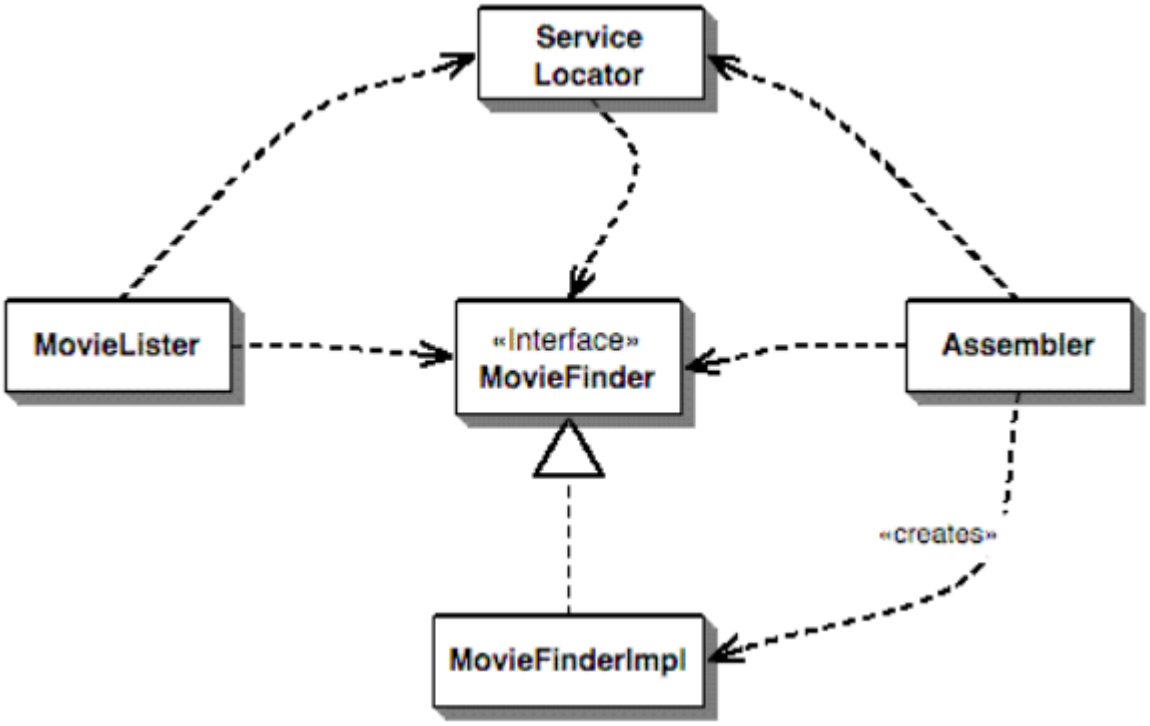
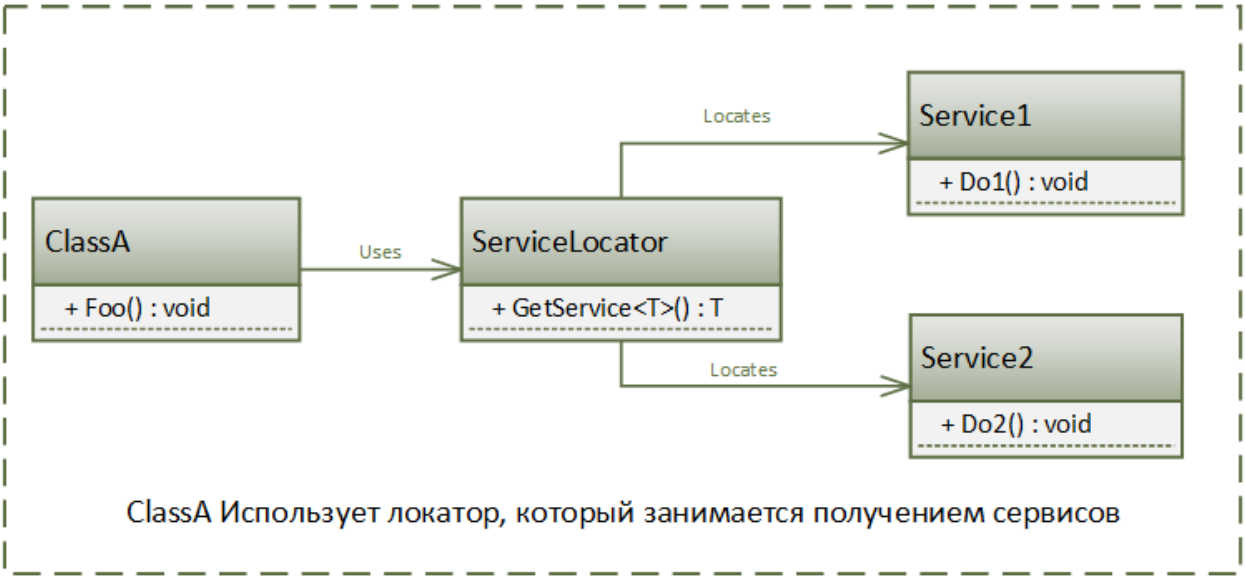
Паттерн Service Locator (SL)

1. **SL:** IoC, Dependency Injection, Service Locator, Мартин Фаулер, 2004, https://martinfowler.com/articles/injection.html#UsingAServiceLocator

2. **SL:** Поисковик фильмов (Мартин Фаулер)

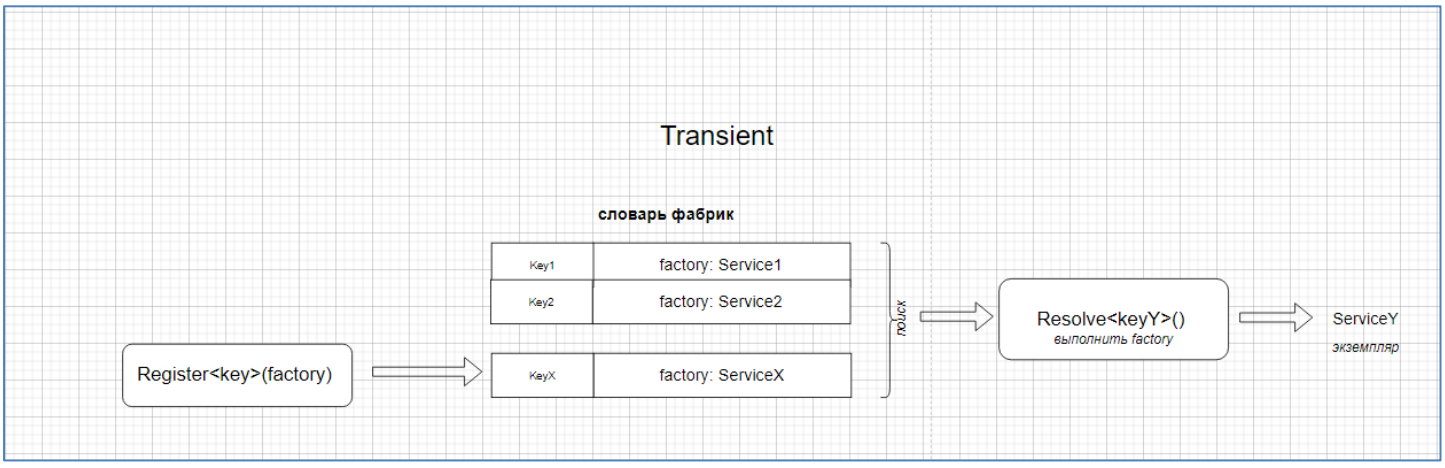


3. **SL:** Локатор сервисов (служб): паттерн проектирования, предназначенный для каталогизации программных объектов (сервисов) с заданным жизненным циклом, а также создания механизма их применения (внедрения) в программном коде. Локатор сервисов отвечает за хранение объектов (с заданным жизненным циклом) и предоставление к ним доступа.



4. **SL:** Анти-паттерн
   https://habr.com/ru/companies/otus/articles/694458/

5. **SL:** Жизненные циклы: ***Transient*** (новый экземпляр при каждом получении), ***Singleton*** (один и тот же экземпляр при каждом получении), ***Scoped*** (один и тот же экземпляр при получении в одном Scope, разные экземпляры в разных Scope).
6. **SL:** ***Transient*** *(разные экземпляры)*

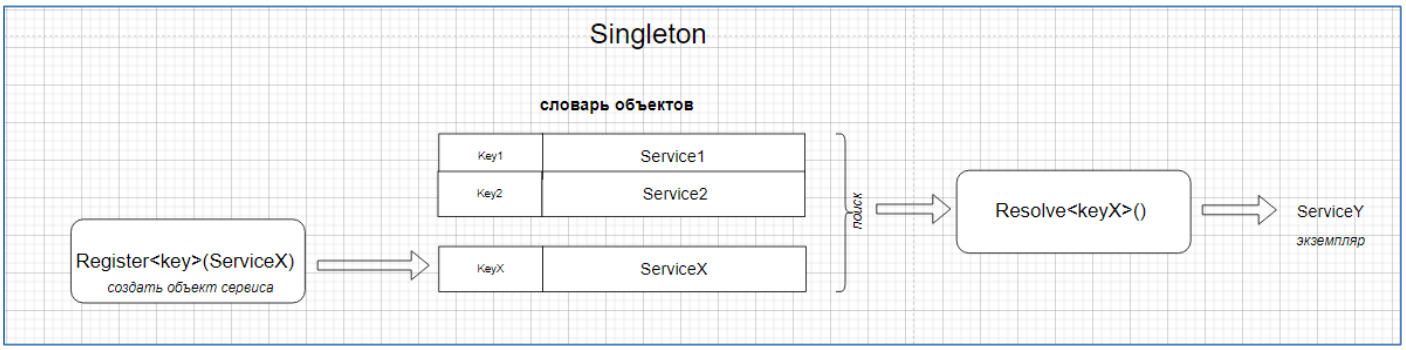



```
TransientLocator.Register<IRepository>(() => MSSQL_DAL.Repository.Create());
//TransientLocator.Register<IRepository>(() => JSON_DAL.Repository.Create());
```

```
using (IRepository repo = TransientLocator.Resolve<IRepository>())
{

    repo.getAllWSRef().ForEach(wsRef => {
        Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
    });
    //.....................................
}
```

```
using (IRepository repo = TransientLocator.Resolve<IRepository>())
{

    repo.getAllWSRef().ForEach(wsRef => {
        Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
    });
    //.....................................
}
```

7. SL: ***Singleton*** *(один и тот же экземпляр)*

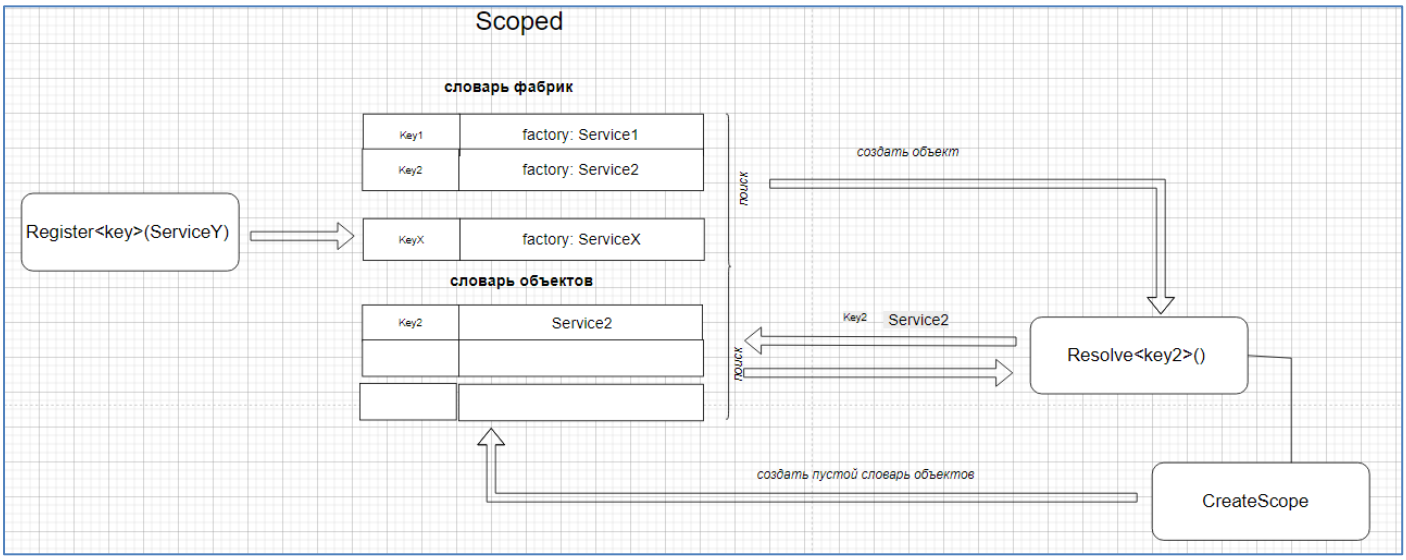



```
SingletonLocator.Register<IRepository>(JSON_DAL.Repository.Create());
// SingletonLocator.Register<IRepository>(MSSQL_DAL.Repository.Create());
```

```
using (IRepository repo = SingletonLocator.Resolve<IRepository>())
{

    repo.getAllWSRef().ForEach(wsRef => {
        Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
    });
    //.....................................
}
```

```
using (IRepository repo = SingletonLocator.Resolve<IRepository>())
{

    repo.getAllWSRef().ForEach(wsRef => {
        Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
    });
    //.....................................
}
```

8. SL: *Scoped*




```
ScopeLocator.Register<IRepository>(() => MSSQL_DAL.Repository.Create());
//ScopeLocator.Register<IRepository>(() => JSON_DAL.Repository.Create());
```

```
IServiceScopeFactory scopeFactory =  ScopeLocator.CreateServiceScopeFactory();

using (IServiceScope scope  =  scopeFactory.CreateScope())
{
    {
        IRepository? repo = scope.ServiceProvider.GetService<IRepository>();
        if (repo != null)
        {
            repo.getAllWSRef().ForEach(wsRef => {
                Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
            });
            // ....................................
        }
    }
    {
        IRepository? repo = scope.ServiceProvider.GetService<IRepository>();
        if (repo != null)
        {
            repo.getAllWSRef().ForEach(wsRef => {
                Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
            });
            //....................................
        }
    }
};
using (IServiceScope scope = scopeFactory.CreateScope())
{
    {
        IRepository? repo = scope.ServiceProvider.GetService<IRepository>();
        if (repo != null)
        {
            repo.getAllWSRef().ForEach(wsRef => {
                Console.WriteLine($"{wsRef.Id}: {wsRef.Url}, {wsRef.Description}, {wsRef.Minus}, {wsRef.Plus}");
            });
            // ....................................
        }
    }
};
```

9. SL: *Transient реализация*

```
//TransientLocator.Register<IRepository>(() => MSSQL_DAL.Repository.Create());
//TransientLocator.Register<IRepository>(() => JSON_DAL.Repository.Create());

public static  class TransientLocator
{
   private readonly static Dictionary<Type, Func<object>>  services = new Dictionary<Type, Func<object>>();

   public static void Register<T>(Func<T> resolver) where T : class
   {
      TransientLocator.services[typeof(T)] = () => resolver();
   }
   public static T Resolve<T>()
   {
       return (T)TransientLocator.services[typeof(T)]();
   }
   public static void Reset()
   {
       TransientLocator.services.Clear();
   }
}
```

10. SL: *Singleton реализация*

```
//SingletonLocator.Register<IRepository>(JSON_DAL.Repository.Create());
//SingletonLocator.Register<IRepository>(MSSQL_DAL.Repository.Create());

public static class SingletonLocator
{
    private readonly static Dictionary<Type, object> services = new Dictionary<Type,object>();

    public static void Register<T>(T service) where T : class
    {
        SingletonLocator.services[typeof(T)] = service;
    }
    public static T Resolve<T>()
    {
        return (T)SingletonLocator.services[typeof(T)];
    }
    public static void Reset()
    {
         SingletonLocator.services.Clear();
    }
}
```

11. SL: *Scoped реализация*

```
// ScopeLocator.Register<IRepository>(() => MSSQL_DAL.Repository.Create());
// ScopeLocator.Register<IRepository>(() => JSON_DAL.Repository.Create());

public static class ScopeLocator
{
    public static IServiceScopeFactory CreateServiceScopeFactory()
    {
        return new ServiceScopeFactory(services);
    }

    private readonly static Dictionary<Type, Func<object>> services = new Dictionary<Type, Func<object>>();

    public static void Register<T>(Func<T> resolver) where T : class
    {
        ScopeLocator.services[typeof(T)] = () => resolver();
    }
}
```

```
// using Microsoft.Extensions.DependencyInjection; // IServiceScopeFactory, IServiceScope, IServiceProvider

//  IServiceScopeFactory scopeFactory =  ScopeLocator.CreateServiceScopeFactory();

public class ServiceScopeFactory : IServiceScopeFactory
{
    private readonly Dictionary<Type, Func<object>> services;
    public  ServiceScopeFactory(Dictionary<Type, Func<object>> services)
    {
        this.services = services;
    }

    public   IServiceScope CreateScope()
    {
       return new ServiceScope(this.services);
    }
}
```

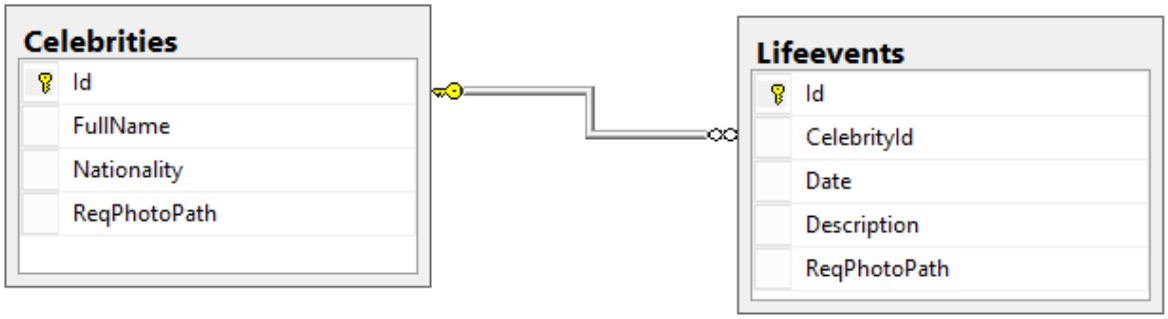
```
//using (IServiceScope scope  =  scopeFactory.CreateScope())

public class ServiceScope : IServiceScope
{
    private Dictionary<Type, Func<object>> services;
    private Dictionary<Type, object> scopeservices;
    public  IServiceProvider ServiceProvider { get; private set;}
    public  ServiceScope(Dictionary<Type, Func<object>>  services)
    {
        this.services = services;
        this.scopeservices =   new Dictionary<Type, object>();
        this.ServiceProvider = new ServiceProvider(this.services, this.scopeservices);
    }
    public void Dispose() {this.scopeservices.Clear(); }
}
```

```
// IRepository? repo = scope.ServiceProvider.GetService<IRepository>();

public class ServiceProvider : IServiceProvider
{
    private Dictionary<Type, Func<object>> services;
    private Dictionary<Type, object> scopeservices;
    public ServiceProvider(Dictionary<Type, Func<object>> services, Dictionary<Type, object> scopeservices)
    {
        this.services = services;
        this.scopeservices = new Dictionary<Type, object>();
    }
    public object? GetService<T>() where T : class
    {
        object? obj = GetService(typeof(T));
        return  obj == null?null:(T)obj;
    }
    public object? GetService(Type serviceType)
    {
        object? obj = this.scopeservices.GetValueOrDefault(serviceType);
        if (obj == null)   obj = this.scopeservices[serviceType] = this.services[serviceType]();
        return obj;
    }
}
```

12. К заданию лабораторной работы:

```
IServiceLocator locator =  builder.AddTransient<IRepository>(() => MSSQL_DAL.Repository.Create())
                                  .AddTransient<IService1>(() => new TestServices1())
                                  .AddSingleton<IService2>(new TestServices2())
                                  .AddScope<IService3>(() => new TestServices3())
                                  .AddSingleton<IService4>(new TestServices4())
                                  .AddScopeFactory()
                                  .Build();


 IRepository repo     =  locator.GetService<IRepository>();
 IService2   service2 =  locator.GetService<IService2>();
 IService4   service4 =  locator.GetService<IService4>();
 IServiceScopeFactory factory = locator.GetService<IServiceScopeFactory>();
 using (IServiceScope scope = factory.CreateScope())
 {
     IService3? service3 = scope.ServiceProvider.GetService<IService3>();
 }
```

13. К заданию лабораторной работы:




```
using (IRepository repo = new Repository())
{
    {
        Celebrity? c = repo.GetCelebrityByLifeeventId(1);
        if (c != null) Console.WriteLine(printC(c));
        else Console.WriteLine("Not Found Id = 1");
    }
    {
        try { repo.AddCelebrity(new Celebrity { FullName = "Edgars Codd", Nationality = "US", ReqPhotoPath = "Codd.jpeg" }); }
        catch (Exception e) { Console.WriteLine(e); }
    }
    {
        try { repo.AddCelebrity(new Celebrity { FullName = " Donald Knuth", ReqPhotoPath = "Codd.jpeg" }); }
        catch (Exception e) { Console.WriteLine(e); }
    }
}
using (ICelebrity repo = new Repository())
{
    repo.GetAllCelebrities().ForEach(celebrity => Console.WriteLine(printC(celebrity)));
    try { repo.AddCelebrity(new Celebrity { FullName = " Donald Knuth", ReqPhotoPath = "Codd.jpeg" }); }
    catch (Exception e) { Console.WriteLine(e); }
 }
using (ILifeevent repo = new Repository())
{
    repo.GetAllLifeevents().ForEach(life => Console.WriteLine($"Id = {life.Id}, CelebrityId = {life.CelebrityId}, " +
                                                              $"Date = {life.Date}, Description = {life.Description}")
    );
}
```

```
public interface ICelebrity<T> : IDisposable
{
    List<T> GetAllCelebrities();                    // получить все Знаменитости
    T? GetCelebrityById(int Id);                    // получить Знаменитость по Id
    bool DelCelebrity(int id);                      // удалить Знаменитость по Id
    bool AddCelebrity(T celebrity);                 // добавить Знаменитость
    bool UpdCelebrity(int id, T celebrity);         // изменить Знаменитость по  Id
}
public interface ILifeevent<T> : IDisposable
{
    List<T> GetAllLifeevents();                     // получить все События
    T? GetLifeevetById(int Id);                     // получить Событие по Id
    bool DelLifeevent(int id);                      // удалить Событие  по Id
    bool AddLifeevent(T lifeevent);                 // добавить Событие
    bool UpdLifeevent(int id, T lifeevent);         // изменить обытие по  Id
}
public class Celebrity  //  Знаменитость
{
    public Celebrity() { this.FullName = ""; this.Nationality = "XX"; }
    public int Id { get; set; }                         // Id Знаменитости
    public string FullName       { get; set; }          // полное имя   Знаменитости
    public string Nationality    { get; set; }          // гражданство  Знаменитости
    public string? ReqPhotoPath  { get; set; }          // reguest path  Фотографии
}
public class Lifeevent  //  Событие в  жизни знаменитости
{
    public Lifeevent() { this.Description = ""; }
    public int Id { get; set; }                          // Id События
    public int CelebrityId      { get; set; }            // Id Знаменитости
    public DateTime Date        { get; set; }            // дата События
    public string Description   { get; set; }            // описание События
    public string? ReqPhotoPath { get; set; }            // reguest path  Фотографии
}
```

```
public interface IRepository: ICommon, ICelebrity, ILifeevent { }

public interface ICelebrity: ICelebrity<Celebrity> { }
public interface ILifeevent: ILifeevent<Lifeevent> { }
public interface ICommon   : ICommon<Celebrity,Lifeevent>{ }

public  interface ICommon<T1, T2>      //: ICelebrity<T1>, ILifeevent<T2>
{
   List<T2> GetLifeeventsByCelebrityId(int celebrityId);  // получить все События по Id Знаменитости
   T1 GetCelebrityByLifeeventId(int lifeeventId);         // получить Знаменитость по Id События
}
```

14. **КОНЕЦ**